

Smart College Hub Backend Viva Questions

Question list with direct answers for college backend presentation preparation.

Project Overview

1. What is Smart College Hub?

Answer: Smart College Hub is a backend-supported college platform that helps students use features such as authentication, coding challenges, and collaboration in one system.

2. What problem does your project solve?

Answer: The project solves the problem of scattered student activities by keeping learning, coding practice, and collaboration features in a centralized application.

3. What are the main backend modules?

Answer: Auth, token, coding challenges, collaborations, join requests, and memberships.

4. Why did you choose FastAPI?

Answer: FastAPI was chosen because it is fast, supports async programming, provides automatic API documentation, validates data using type hints, and is simple to structure.

5. What is the role of app/main.py?

Answer: It creates the FastAPI app, adds middleware, and registers all routers.

6. How is the backend connected to the frontend?

Answer: Frontend calls REST APIs exposed by FastAPI using HTTP requests.

7. What is a router in FastAPI?

Answer: A router groups related endpoints so the code stays organized.

8. Why did you separate routes, services, repositories, models, and schemas?

Answer: Separation makes the code maintainable, testable, and easier to explain.

9. Explain the request flow in your backend.

Answer: Request goes to route, route calls service, service calls repository, repository uses database, then response is returned.

10. What future improvements can be added?

Answer: Testing, deployment, rate limiting, better logging, notifications, and production security hardening.

FastAPI

11. What is FastAPI?

Answer: A modern Python framework for building APIs quickly with automatic validation and docs.

12. What are the benefits of FastAPI?

Answer: High performance, async support, type hints, Swagger docs, and dependency injection.

13. What is an endpoint?

Answer: A URL path plus HTTP method that performs a backend action.

14. What is include_router() used for?

Answer: It attaches route groups to the main FastAPI app.

15. What are tags in FastAPI?

Answer: Tags organize endpoints in the Swagger documentation.

16. What is Swagger UI?

Answer: The automatic interactive API documentation available in the browser.

17. What is dependency injection?

Answer: FastAPI automatically provides required objects like database sessions or current users.

18. What is Depends()?

Answer: It declares a dependency that FastAPI should run before the endpoint.

19. Why use async functions?

Answer: They help handle database and network operations efficiently.

20. What is middleware?

Answer: Code that runs before or after requests, such as CORS middleware.

Authentication

21. How does registration work?

Answer: The backend validates user data, hashes the password, and stores the user.

22. How does login work?

Answer: The backend checks email/username and password, then returns tokens if valid.

23. Why do we hash passwords?

Answer: So original passwords are not stored and cannot be easily stolen.

24. Which hashing library is used?

Answer: Passlib with bcrypt.

25. What is bcrypt?

Answer: A secure password hashing algorithm designed to be slow against brute-force attacks.

26. How do you verify passwords?

Answer: Compare the plain password with the stored hash using `verify_password()`.

27. What happens if credentials are invalid?

Answer: The backend returns an authentication error instead of a token.

28. What is the role of AuthService?

Answer: It contains authentication business logic.

29. What is the role of UserRepository?

Answer: It handles user-related database operations.

30. Why not store password in plain text?

Answer: It is a serious security risk if the database is leaked.

JWT And Tokens

31. What is JWT?

Answer: A signed token used to securely transfer user identity and claims.

32. Why did you use JWT?

Answer: It allows stateless authentication between frontend and backend.

33. What data is inside your token?

Answer: uid, email, username, role, expiry, jti, and refresh flag.

34. What is an access token?

Answer: A short-lived token used to access protected APIs.

35. What is a refresh token?

Answer: A longer-lived token used to generate a new access token.

36. Difference between access and refresh token?

Answer: Access token is for API access; refresh token is for renewing access.

37. Why does the access token expire?

Answer: To reduce damage if the token is stolen.

38. What is jti?

Answer: A unique token ID used for tracking and blacklisting tokens.

39. Why is uid validated as UUID?

Answer: To ensure token identity format is valid and safe.

40. What happens when a token expires?

Answer: The backend returns 401 Unauthorized.

41. What happens when a token is invalid?

Answer: The backend rejects it with an invalid token error.

42. How does logout work?

Answer: The token can be added to a blacklist so it cannot be reused.

43. Why use Redis for token blacklist?

Answer: Redis is fast and good for temporary token storage.

44. What is the JWT secret?

Answer: A private key used to sign and verify tokens.

45. Why store JWT secret in .env?

Answer: Secrets should not be hardcoded in source code.

Authorization And Roles

46. What is authentication?

Answer: Checking who the user is.

47. What is authorization?

Answer: Checking what the user is allowed to do.

48. Difference between authentication and authorization?

Answer: Authentication verifies identity; authorization verifies permission.

49. What is role-based access control?

Answer: Access is allowed based on user roles such as student or admin.

50. How does RoleChecker work?

Answer: It checks if the current user's role is inside allowed_roles.

51. What happens if a user has no permission?

Answer: The backend returns 403 Forbidden.

52. Why protect routes?

Answer: To prevent unauthorized users from accessing private data or actions.

53. Where is the current user retrieved?

Answer: From token data using get_current_user().

Database

54. Which database layer is used?

Answer: SQLAlchemy with SQLAlchemy async session.

55. What is SQLAlchemy?

Answer: A library combining Pydantic-style validation with SQLAlchemy ORM features.

56. What is an ORM?

Answer: Object Relational Mapper; it maps Python classes to database tables.

57. Why use ORM?

Answer: It makes database operations easier and safer than raw SQL for common tasks.

58. What is AsyncSession?

Answer: An async database session used for non-blocking database operations.

59. What does get_session() do?

Answer: It creates and yields a database session for each request.

60. What is DATABASE_URL?

Answer: The connection string used to connect the backend to the database.

61. What is a model?

Answer: A Python class that represents a database table.

62. What is a schema?

Answer: A validation and serialization structure for request and response data.

63. Difference between model and schema?

Answer: Model maps to database; schema controls API data shape.

64. What is a primary key?

Answer: A unique identifier for each row in a table.

65. What is a foreign key?

Answer: A field that links one table to another table.

66. What is cascade delete?

Answer: Related child records are deleted when the parent record is deleted.

67. Why use indexes?

Answer: Indexes speed up searches and filtering on important columns.

Alembic Migrations

68. What is Alembic?

Answer: A database migration tool for SQLAlchemy projects.

69. Why are migrations needed?

Answer: They track and apply database structure changes safely.

70. What is a migration file?

Answer: A versioned script that changes the database schema.

71. Why not manually edit database tables?

Answer: Manual changes are hard to track and can break team consistency.

72. How do migrations help deployment?

Answer: They let production database schema match the application code.

73. What happens when a model changes?

Answer: A new migration should be created and applied.

74. Why keep migration files in Git?

Answer: They document database history with the codebase.

CORS

75. What is CORS?

Answer: A browser security rule controlling cross-origin frontend-backend requests.

76. Why did you add CORS middleware?

Answer: To allow the frontend running on localhost ports to call the backend.

77. What happens without CORS?

Answer: The browser may block frontend API requests.

78. Why list specific localhost origins?

Answer: It allows development frontend URLs without opening the API to every origin.

79. What does `allow_methods=['*']` mean?

Answer: All HTTP methods like GET, POST, PUT, DELETE are allowed.

80. What does `allow_headers=['*']` mean?

Answer: All request headers are allowed.

81. What does `allow_credentials=True` mean?

Answer: Credentials such as auth headers can be included in requests.

82. Should all origins be allowed in production?

Answer: No, production should allow only trusted domains.

Collaboration Module

83. What is the collaboration feature?

Answer: It lets users create, join, and manage collaboration groups/projects.

84. Why separate collaboration routes?

Answer: To keep collaboration logic organized and independent.

85. What is a join request?

Answer: A request from a user to join a collaboration.

86. What is membership?

Answer: A record showing that a user belongs to a collaboration.

87. Difference between owner and member?

Answer: Owner manages the collaboration; member participates in it.

88. How do you prevent duplicate membership?

Answer: Using service checks or database constraints.

89. How are join requests approved?

Answer: The owner/admin accepts the request and membership is created.

90. What happens when a collaboration is deleted?

Answer: Related memberships and requests can be removed through cascade rules.

91. Why have separate join request and membership tables?

Answer: A request is pending approval; membership means the user has joined.

92. What validation is needed in collaboration APIs?

Answer: Check ownership, membership status, duplicate requests, and permissions.

Coding Challenge Module

93. What is the coding challenge module?

Answer: It manages coding problems and student submissions.

94. Who can create challenges?

Answer: Challenge creation should be limited to authorized users such as admins or teachers so students cannot modify official challenge content.

95. What is a submission?

Answer: A student's answer or code submitted for a challenge.

96. Why store submission feedback?

Answer: So students can see evaluation comments or results.

97. Why add submission indexes?

Answer: To make submission searches and filters faster.

98. How does the backend validate challenge data?

Answer: Using schemas and service-layer checks.

99. What endpoints are expected for challenges?

Answer: Create, list, detail, update, delete, and submit.

100. What improvement could be added?

Answer: Automatic code execution, test cases, leaderboard, and plagiarism checks.

Repository And Service Pattern

101. What is repository pattern?

Answer: A layer that handles database queries separately from business logic.

102. What is a service layer?

Answer: A layer that contains business rules and coordinates repositories.

103. Why not write everything in route files?

Answer: Routes become messy and hard to test if they contain all logic.

104. What belongs in routes?

Answer: Request/response handling, dependencies, and calling services.

105. What belongs in services?

Answer: Business logic such as validation, permissions, and workflows.

106. What belongs in repositories?

Answer: Database create, read, update, delete operations.

107. How does this structure improve maintainability?

Answer: Each file has one responsibility and changes are easier to isolate.

108. How does this help testing?

Answer: Services and repositories can be tested separately.

Errors And Status Codes

109. What is HTTPException?

Answer: FastAPI's way to return controlled HTTP errors.

110. When do you return 400?

Answer: Bad request or invalid input.

111. When do you return 401?

Answer: User is not authenticated or token is invalid.

112. When do you return 403?

Answer: User is authenticated but not allowed.

113. When do you return 404?

Answer: Requested resource was not found.

114. When do you return 500?

Answer: Unexpected server error.

115. How are expired tokens handled?

Answer: JWT decode catches expiry and returns 401.

116. How are invalid UUIDs handled?

Answer: The backend rejects them with an authentication error.

117. Why should error messages be clear?

Answer: They help frontend and users understand what went wrong.

Security

118. How is your backend secured?

Answer: Password hashing, JWT auth, role checks, token blacklist, and environment secrets.

119. What if JWT secret is leaked?

Answer: Attackers could create fake valid tokens.

120. Why use .env?

Answer: To keep secrets and environment-specific settings outside source code.

121. Why is refresh token protection important?

Answer: Refresh tokens can create new access tokens.

122. What is token blacklisting?

Answer: Marking a token as invalid before its normal expiry.

123. How can brute force login be reduced?

Answer: Rate limiting, account lockout, captcha, and monitoring.

124. What production security improvements are needed?

Answer: HTTPS, secure cookies, strict CORS, rate limiting, logging, and strong secrets.

125. Why should debug logging be reduced in production?

Answer: Logs can leak sensitive data and reduce performance.

Deployment And Testing

126. How would you deploy the backend?

Answer: The backend can be deployed on a server using Uvicorn or Gunicorn, connected to a production database and Redis, configured with environment variables, and protected using HTTPS.

127. What is Uvicorn?

Answer: An ASGI server used to run FastAPI applications.

128. Development vs production environment?

Answer: Development is local and flexible; production must be secure and stable.

129. Why disable database echo in production?

Answer: It prints SQL queries and may expose sensitive information.

130. How did you test APIs?

Answer: Using Swagger UI, frontend requests, or tools like Postman.

131. What login test cases are important?

Answer: Valid login, wrong password, missing fields, expired token, and logout.

132. What collaboration test cases are important?

Answer: Create collaboration, request join, approve request, duplicate request, and permission checks.

133. How do you debug backend errors?

Answer: Check logs, status codes, request data, database state, and token validity.

Most Important Viva Questions

134. Explain your backend architecture.

Answer: The backend follows a layered architecture where routes receive requests, services handle business logic, repositories handle database operations, models represent tables, and schemas validate API data.

135. Explain login from frontend to backend.

Answer: Frontend sends credentials, backend verifies password, returns JWT tokens.

136. Why did you use JWT?

Answer: For stateless secure authentication.

137. Difference between access token and refresh token?

Answer: Access is short-lived for APIs; refresh is longer-lived for renewal.

138. What is Redis used for?

Answer: Token blacklist storage.

139. What is CORS?

Answer: Browser rule for allowing frontend-backend communication across origins.

140. What are Alembic migrations?

Answer: Version-controlled database schema changes.

141. Explain collaboration module.

Answer: Users create collaborations, send join requests, and become members after approval.

142. Explain coding challenge module.

Answer: It manages challenges and student submissions/feedback.

143. What would you improve next?

Answer: Testing, deployment, notifications, rate limiting, and stronger production security.